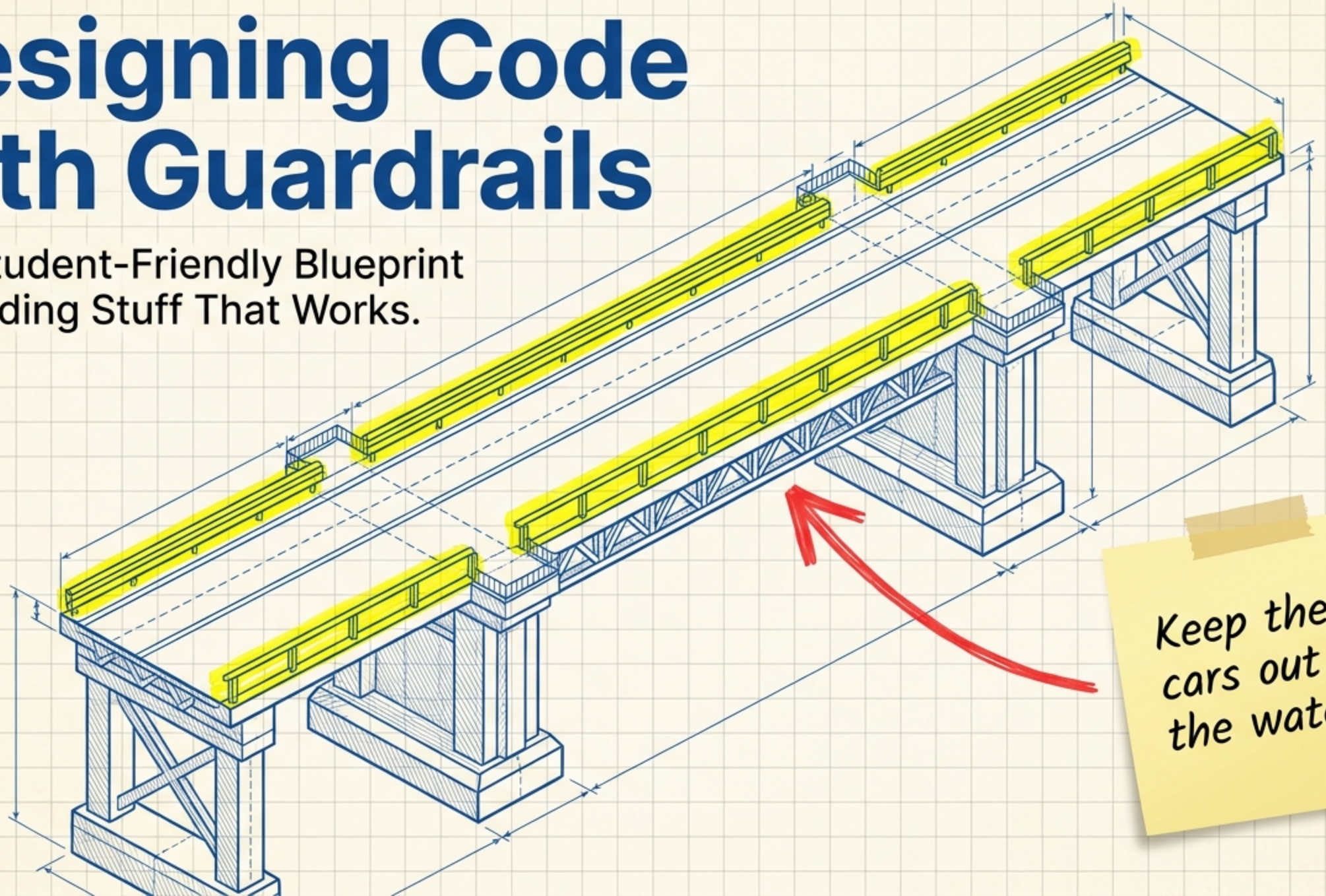
Designing Code with Guardrails

The Student-Friendly Blueprint to Building Stuff That Works.

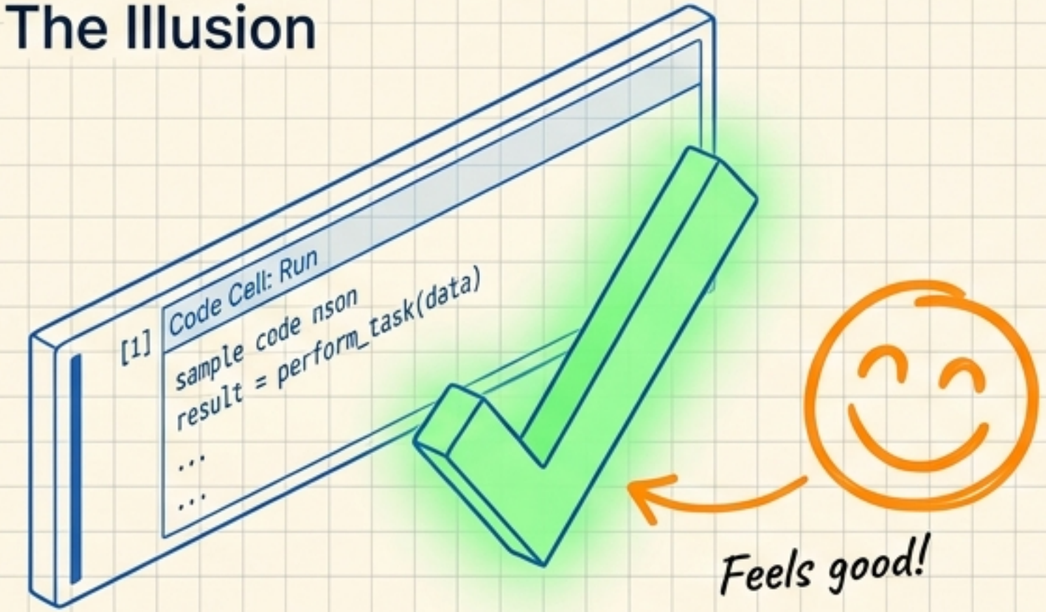


Keep the cars out of the water!

The trap of “Vibe Coding” vs. Engineering

A green checkmark doesn’t mean your code is right.
It just means it didn’t crash.

The Illusion



Code that runs without errors (the green cell in VS Code or Jupyter). Looks finished on the surface.

The Reality

‘Quiet Mistakes’ — silent data corruption that acts like an invisible bug. Scope creeps cell by cell, cells run out of order, and the project is entirely broken underneath.

The Engineering Reality: Done is not ‘it runs.’ Done means the code satisfies a spec, the data is proven clean, and the history proves the work was incremental.

Layer 1: The Personal Workspace Bubble

The .venv (Virtual Environment)

A glowing, protective bubble that shields your project from the chaos of your main computer.

Why do we need it?

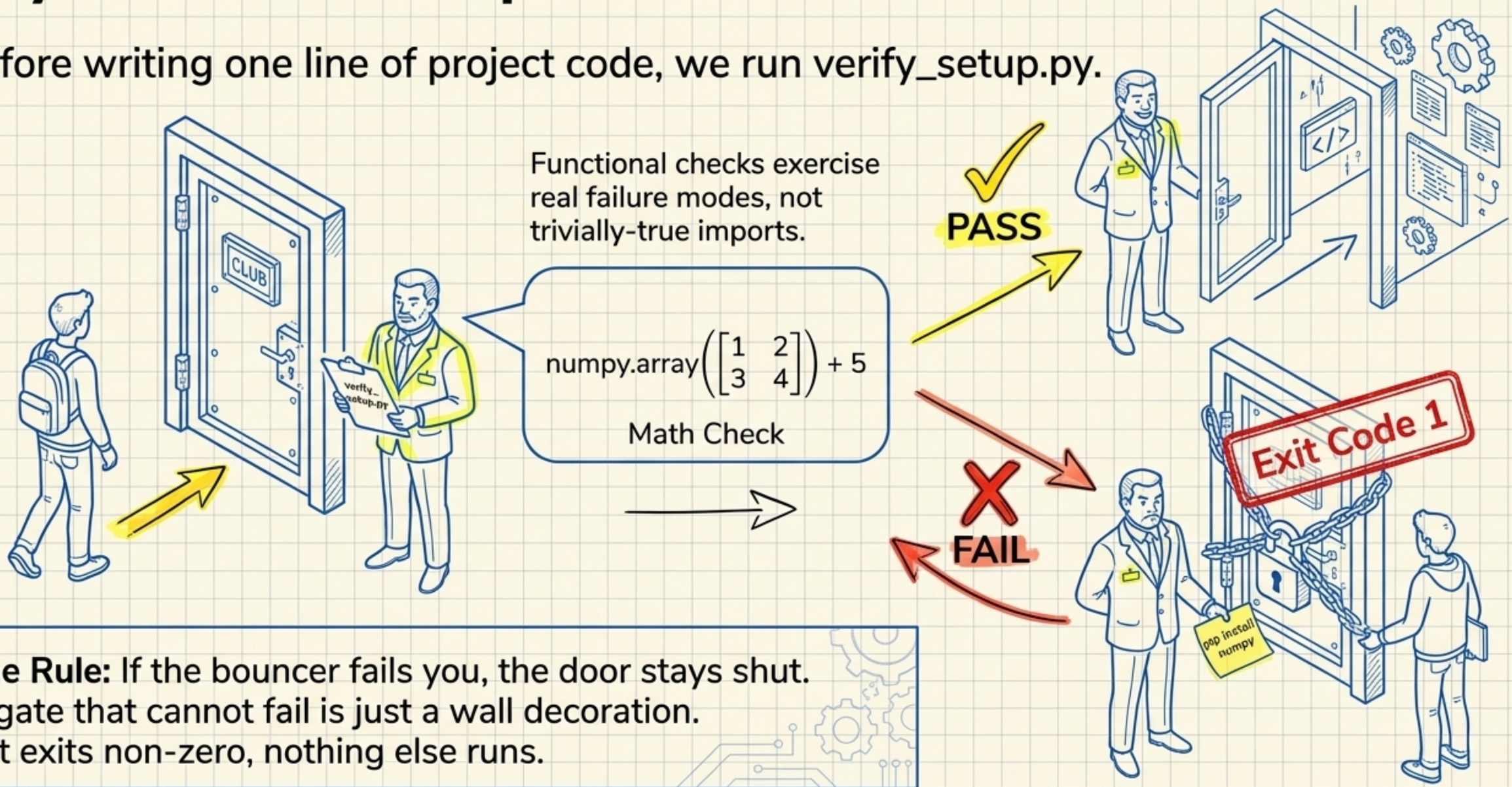
- Cures the "it works on my machine" nightmare.
- Ensures you, your teammates, and your professor are playing with the exact same tools.

Rule of the Bubble:

Every package must be recorded in requirements.txt. No ad-hoc, unrecorded installs allowed inside!

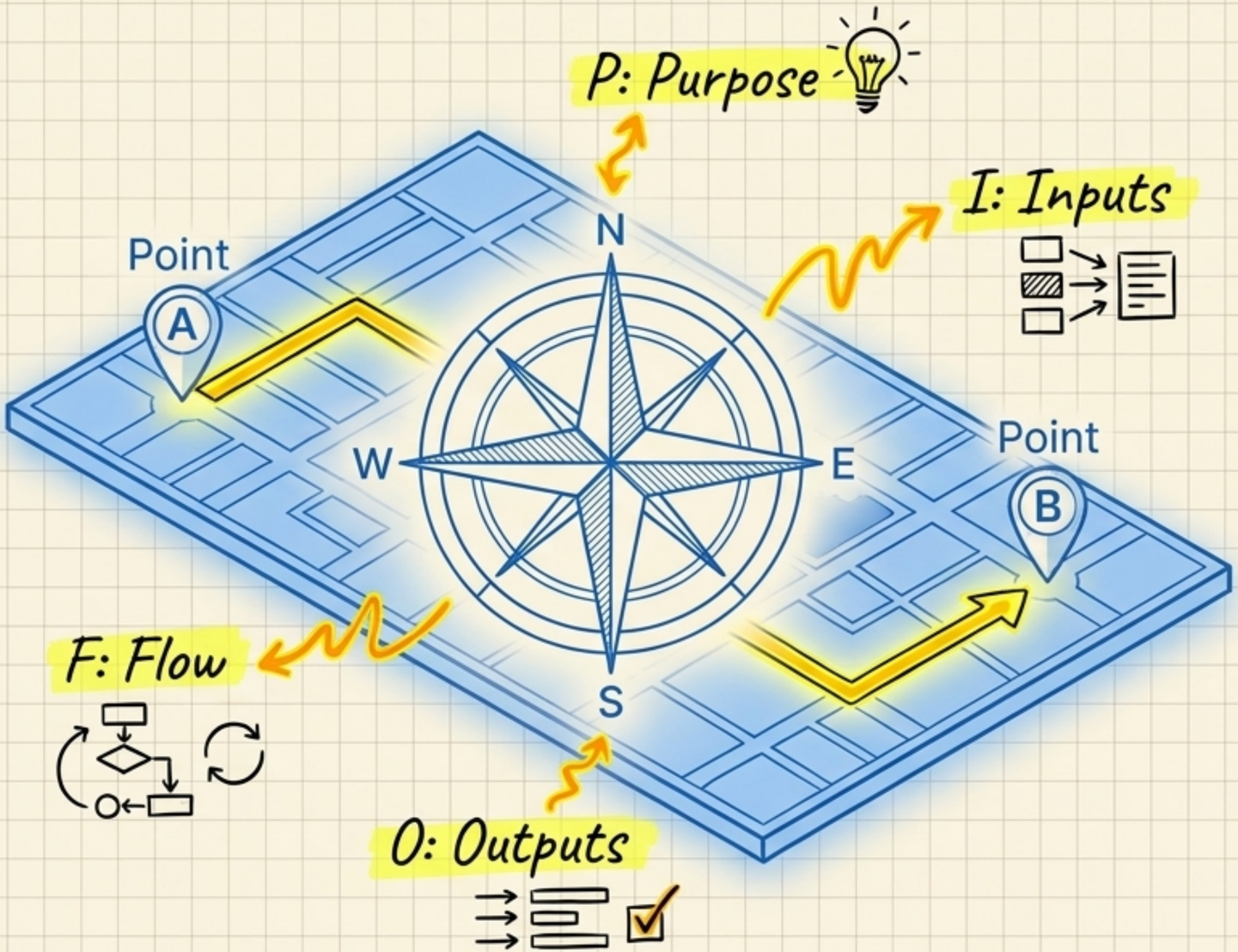
Layer 2: The Setup Bouncer

Before writing one line of project code, we run verify_setup.py.



The Rule: If the bouncer fails you, the door stays shut.
A gate that cannot fail is just a wall decoration.
If it exits non-zero, nothing else runs.

The Developer's Compass: P-I-O-F



Checkpoint C1: The Ultimate Planning Guide

Plan before you type!

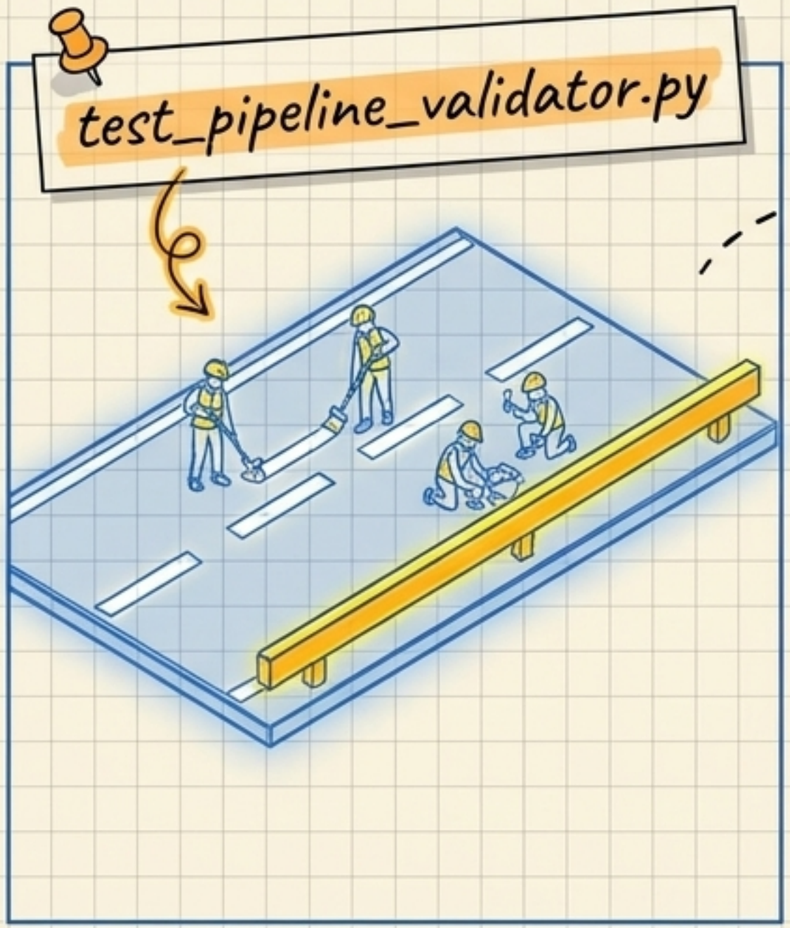
Before writing any code for a new function, state its P-I-O-F.

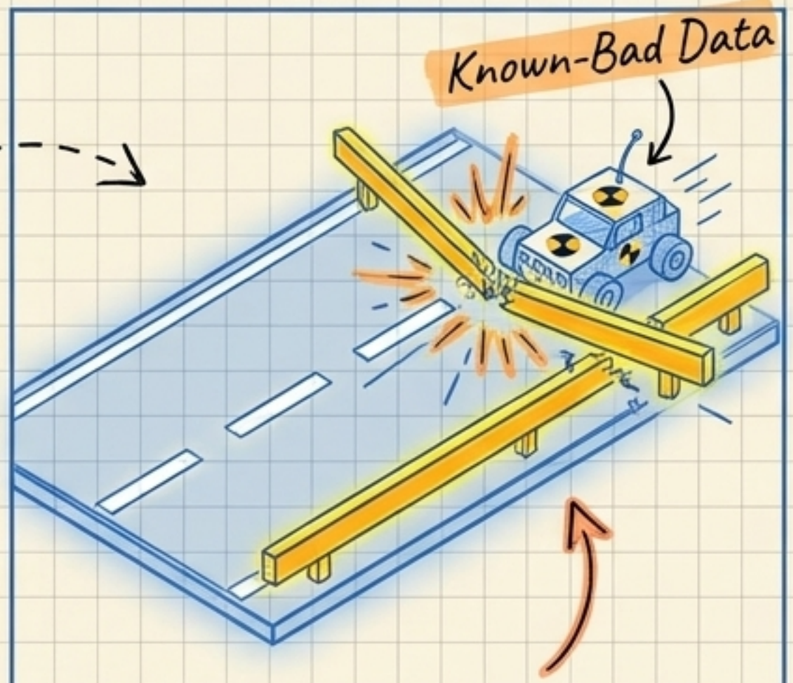
If you cannot explain the route without looking at your notes, you aren’t ready to drive.

Code written without a compass solves the wrong problem.

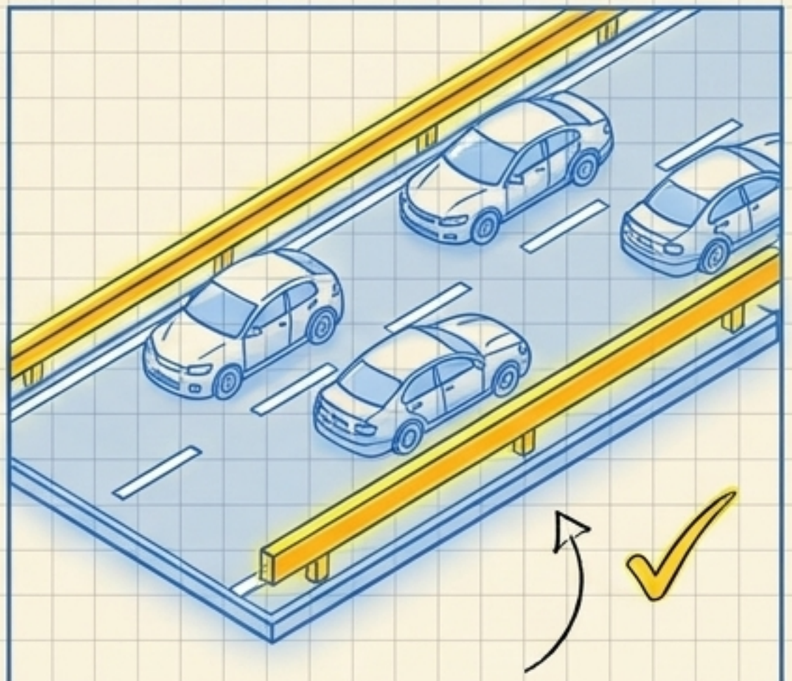
Writing the Rules Before the Game (TDD)

Tests are a contract (written in Gherkin: PASS or FAIL, no partial credit).
It's like drawing lane lines on a highway before letting the cars drive.





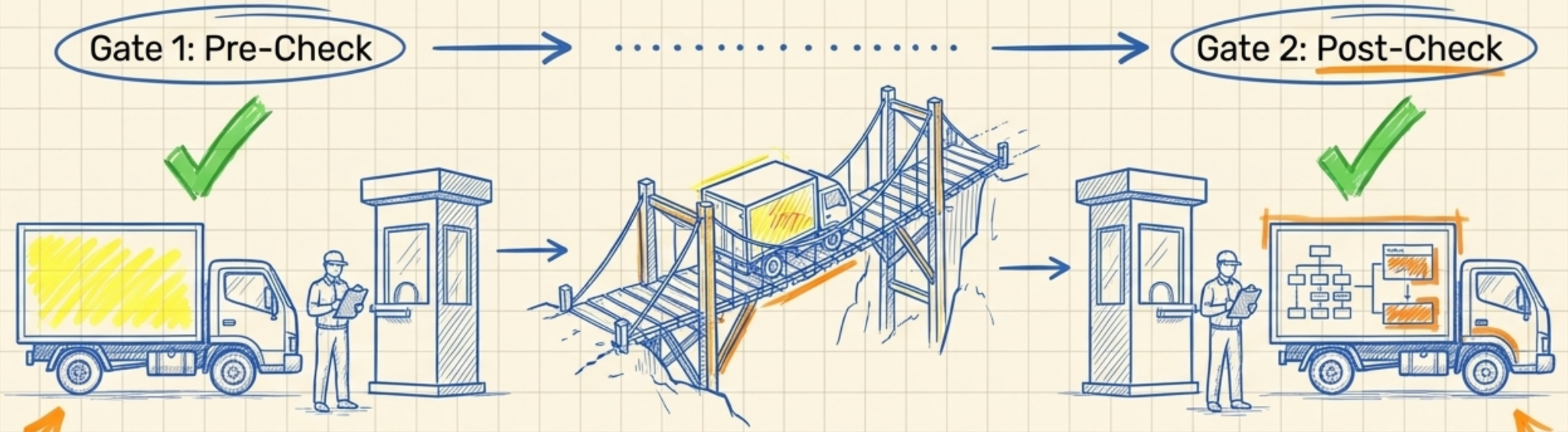
Deliberately feed corrupt data and EXPECT an AssertionError.



Real data flows safely. Silence is ambiguous; a failure message must name the fix.

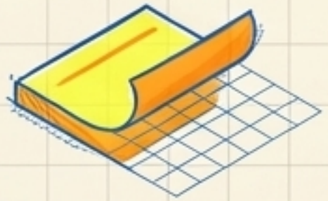
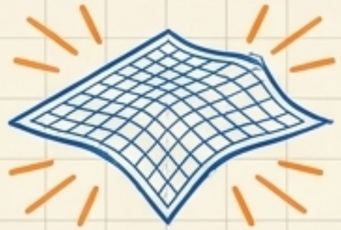
Layer 3: The Double Gateposts

Pre- and Post-Transform Assertions. The validator call is never optional—it wraps the math.

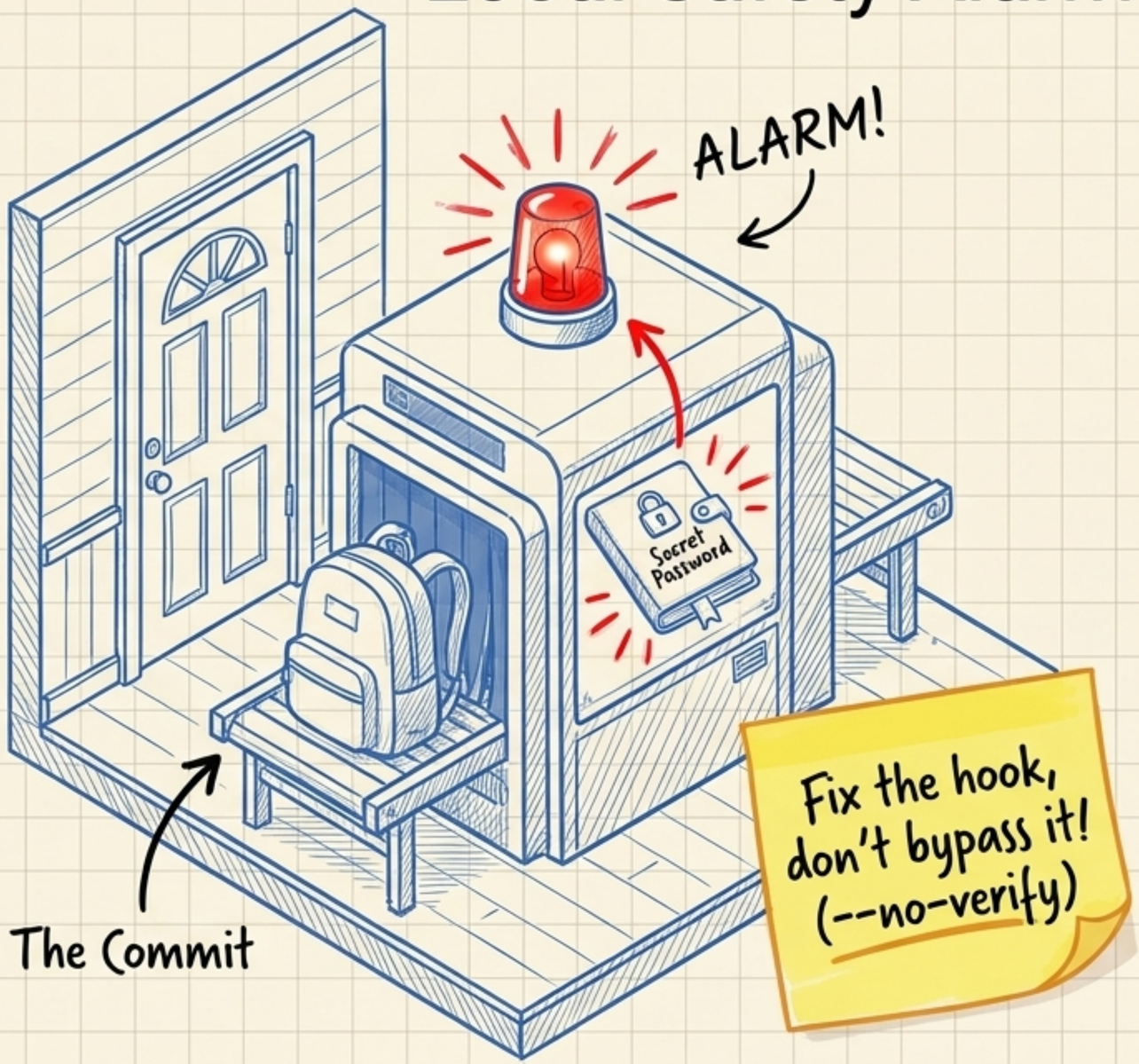


Why? If the truck is fine at Gate 1, but damaged at Gate 2, you know exactly what broke it (the bridge). Without both gates, a failure is entirely ambiguous.

The 4 Quiet Mistakes (And How to Catch Them)

The Mistake	What Happens	The Safety Alarm
 Mismatched Sticky Notes	Pandas pads mismatched rows with empty, invisible gaps (NaNs). *Invisible gaps!*	`check_alignment`
 The Stretching Mistake	Numpy gets confused and stretches math (broadcasting) along the wrong axis. *Math on wrong axis!*	`check_broadcasting` and `keepdims=True`
 Poison in the Code	A single division-by-zero acts like poison, turning downstream numbers into invalid infinity, silently destroying models. *Silently destroys models!*	`check_integrity` via `np.isfinite().all()`
 Broken Boundaries	Features on completely different scales dominate the model. *Different scales dominate!*	`check_scale`

Local Safety Alarms: Git Hooks



Scripts that Git runs automatically on your local machine before letting code leave.

The Pre-Commit Hook

Scans your pockets for passwords or secrets before they leave your computer.

Why Local?

Because once a secret is pushed to GitHub, it is already on the internet and in the receive buffer.

The Golden Rule

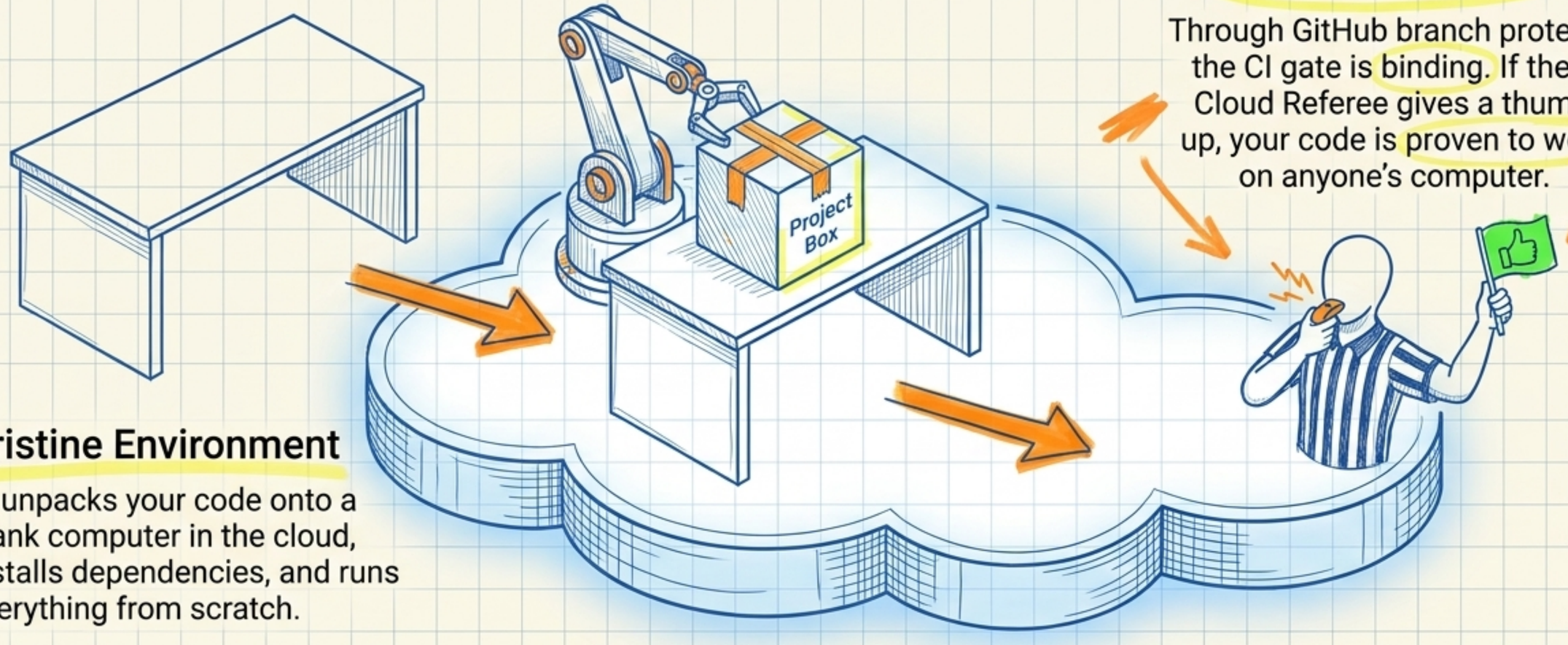
The `--no-verify` command skips these alarms. Never use it just because a test is slow. If a hook is wrong, fix the hook.

The Cloud Referee: GitHub CI/CD Actions

The CI layer re-runs your Phase 0 gates on every push. It adds no new logic.

The Binding Vote

Through GitHub branch protection, the CI gate is binding. If the Cloud Referee gives a thumbs up, your code is proven to work on anyone's computer.



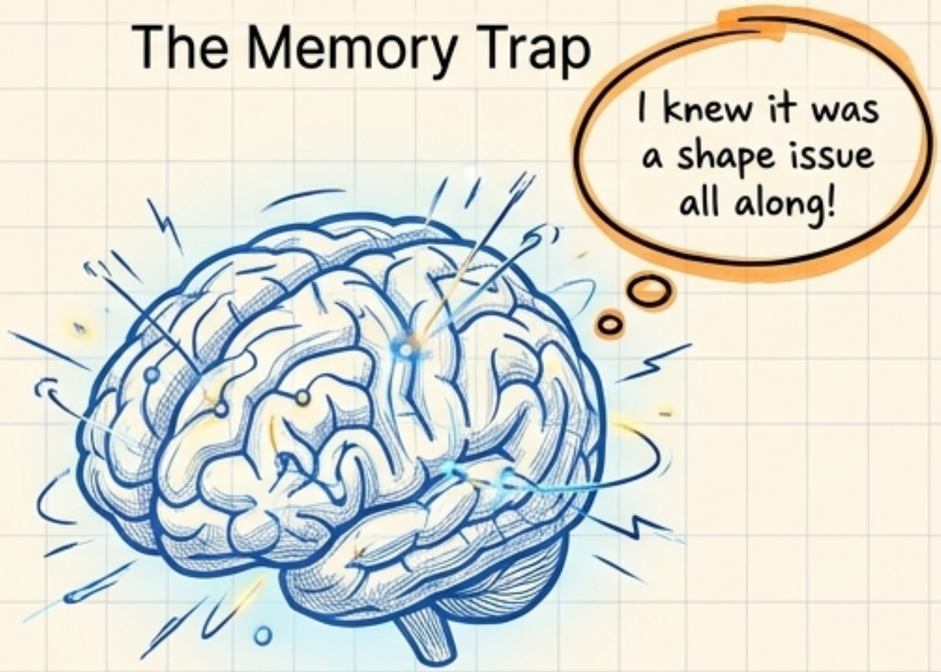
Pristine Environment

CI unpacks your code onto a blank computer in the cloud, installs dependencies, and runs everything from scratch.

Checkpoint C3: The Debugging Odyssey

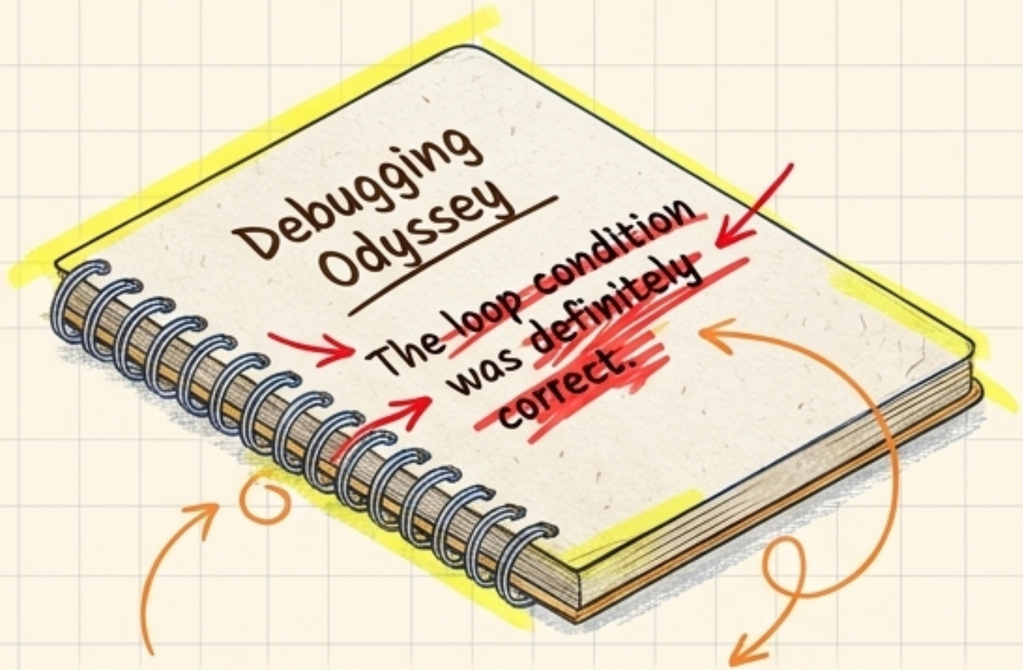
Write down your wrong assumptions BEFORE you fix your code.

The Memory Trap



When something fails, your brain is instantly primed to rewrite its own history. Within an hour of fixing a bug, you will convince yourself you knew the answer all along.

The True Value



You did not know. The wrong assumption is the actual learning moment. Capturing the mental-model correction before touching the code preserves your real progress.

The Complete Blueprint

"Engineering isn't about never making mistakes; it's about building an environment where mistakes are caught for you."